

Documentação Swagger (OpenAPI) e Arquitetura Node.js

1. Objetivo

Este documento complementa a documentação técnica do banco de dados e tem como objetivo: - Padronizar a **API REST** da aplicação - Servir como base para **Swagger / OpenAPI** - Definir uma **arquitetura de pastas Node.js** clara, escalável e profissional

A ideia é permitir que qualquer desenvolvedor Node consiga **subir a API, conectar ao banco SQL Server e integrar o frontend** rapidamente.

2. Padrões adotados

- **Node.js + Express**
- **JWT** para autenticação
- **Camada de serviços** separada da rota
- **Acesso ao banco via procedures e views**
- **Swagger como fonte de verdade da API**

3. Swagger / OpenAPI – Estrutura Geral

3.1 Metadados

openapi: 3.0.3

info:

title: API – Plataforma de Fretes de Retorno

description: API REST para anúncios de fretes de retorno, veículos e mercadorias

version: 1.0.0

servers:

- url: <https://api.fretesretorno.com/v1>

3.2 Segurança (JWT)

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

security:
  - bearerAuth: []
```

4. Schemas (Models da API)

4.1 Usuario

```
Usuario:
  type: object
  properties:
    idUsuario:
      type: integer
    nome:
      type: string
    tipoUsuario:
      type: string
      enum: [C, E]
    email:
      type: string
```

4.2 Veiculo

```
Veiculo:  
  type: object  
  properties:  
    idVeiculo:  
      type: integer  
    placa:  
      type: string  
    tipoVeiculo:  
      type: string  
    capacidadePesoKg:  
      type: number  
    capacidadeVolumeM3:  
      type: number
```

4.3 AnuncioResumo

```
AnuncioResumo:  
  type: object  
  properties:  
    idAnuncio:  
      type: integer  
    tipoAnuncio:  
      type: string  
    origem:  
      type: string  
    destino:  
      type: string  
    tipoVeiculo:  
      type: string  
    tipoCarroceria:  
      type: string  
    refrigerado:  
      type: boolean  
    valorSugerido:  
      type: number
```

5. Endpoints Documentados (Swagger)

5.1 Autenticação

/auth/login:

post:

tags: [Auth]

summary: Login do usuário

requestBody:

required: true

content:

application/json:

schema:

type: object

properties:

email:

type: string

senha:

type: string

responses:

200:

description: Login realizado

5.2 Anúncios – Busca Inteligente

/anuncios/busca:

get:

tags: [Anuncios]

summary: Busca anúncios com filtros

parameters:

- in: query
name: origemCidade
schema:
type: string
- in: query
name: destinoEstado
schema:
type: string
- in: query
name: tipoVeiculo
schema:
type: string

responses:

200:

description: Lista de anúncios

content:

application/json:

schema:

type: array

items:

\$ref: '#/components/schemas/AnuncioResumo'

6. Arquitetura de Pastas – Node.js

6.1 Estrutura recomendada

```
src/  
├── app.js  
├── server.js  
├── config/  
│   ├── database.js  
│   ├── jwt.js  
│   └── swagger.js  
├── modules/  
│   ├── auth/  
│   │   ├── auth.controller.js  
│   │   ├── auth.service.js  
│   │   └── auth.routes.js  
│   ├── usuarios/  
│   │   ├── usuario.controller.js  
│   │   ├── usuario.service.js  
│   │   └── usuario.routes.js  
│   ├── veiculos/  
│   │   ├── veiculo.controller.js  
│   │   ├── veiculo.service.js  
│   │   └── veiculo.routes.js  
│   ├── anuncios/  
│   │   ├── anuncio.controller.js  
│   │   ├── anuncio.service.js  
│   │   └── anuncio.routes.js  
│   └── mercadorias/  
│       ├── mercadoria.controller.js  
│       ├── mercadoria.service.js  
│       └── mercadoria.routes.js  
├── middlewares/  
│   ├── auth.middleware.js  
│   └── error.middleware.js  
├── database/  
│   ├── queries.js  
│   └── procedures.js  
└── utils/  
    ├── logger.js  
    └── response.js
```

Retorno inteligente

7. Responsabilidade por Camada

Controller

- Recebe request
- Valida entrada
- Chama service
- Retorna response HTTP

Service

- Contém regras de negócio
- Chama procedures e views
- Controla transações

Database

- Centraliza SQL
- Executa procedures
- Evita SQL espalhado

8. Integração com SQL Server

- Usar biblioteca `mssql`
- Sempre chamar **Stored Procedures** para buscas
- Usar **Views** para listagens
- Nunca montar SQL dinâmico no controller

9. Swagger no Projeto

Inicialização

- `/config/swagger.js`
- Swagger UI em `/api/docs`

Swagger deve ser usado como: - Documentação oficial - Base para testes - Contrato com frontend
